

EXPERIMENT 12: Named Entity Recognition using Hidden Markov Model

AIM:

To implement a Named Entity Recognition (NER) system using a Hidden Markov Model (HMM) trained on a small annotated dataset with entity tags: PER (Person), ORG (Organization), LOC (Location), and O (Other).

PROCEDURE:

1. Import numpy and defaultdict.
2. Define small annotated training dataset with (word, tag) pairs.
3. Build vocabulary and tag set from training data.
4. Create word2idx, tag2idx mappings.
5. Initialize transition_counts ($T \times T$), emission_counts ($T \times V$), tag_counts (T).
6. Count tag transitions, word emissions, and tag frequencies.
7. Compute transition probabilities with Laplace smoothing.
8. Compute emission probabilities with Laplace smoothing.
9. Implement Viterbi decoding to find the most likely tag sequence.
10. Test with sentence "Bill visited California", print predicted NER tags.

CODE:

```
import numpy as np from collections
import defaultdict
data = [
    [("Barack", "PER"), ("Obama", "PER"), ("visited", "O"), ("India", "LOC")],
    [("Google", "ORG"), ("is", "O"), ("in", "O"), ("California", "LOC")],

    [("Microsoft", "ORG"), ("was", "O"), ("founded", "O"), ("by", "O"), ("Bill", "PER"), ("Gates", "PER")],
]
words = set(); tags = set()
for sentence in data:
    for word, tag in sentence:
        words.add(word); tags.add(tag)
words = list(words); tags = list(tags)
word2idx = {w:i for i,w in enumerate(words)}
tag2idx = {t:i for i,t in enumerate(tags)}
idx2tag = {i:t for t,i in tag2idx.items()} V
= len(words); T = len(tags)
transition_counts = np.zeros((T,T))
emission_counts = np.zeros((T,V))
```

```
tag_counts = np.zeros(T)

for sentence in data:
    prev_tag = None
    for word, tag in sentence:
        t = tag2idx[tag]; w = word2idx[word]
        emission_counts[t][w] += 1
        tag_counts[t] += 1
        if prev_tag is not None:
            transition_counts[tag2idx[prev_tag]][t] += 1
        prev_tag = tag

transition_probs = (transition_counts + 1) / (tag_counts.reshape(-1,1) + T)
emission_probs = (emission_counts + 1) / (tag_counts.reshape(-1,1) + V)

def viterbi_ner(sentence):
    tokens = sentence.split()
    n = len(tokens)
    delta = np.zeros((n, T))
    psi = np.zeros((n, T), dtype=int)
    for t in range(T):
        w_idx = word2idx.get(tokens[0], -1)
        em = emission_probs[t][w_idx] if w_idx >= 0 else 1/(tag_counts[t]+V)
        delta[0][t] = (1/T) * em
    for i in range(1, n):
        for j in range(T):
            w_idx = word2idx.get(tokens[i], -1)
            em = emission_probs[j][w_idx] if w_idx >= 0 else 1/(tag_counts[j]+V)
            vals = delta[i-1] * transition_probs[:,j]
            psi[i][j] = np.argmax(vals)
            delta[i][j] = np.max(vals) * em
    path = np.zeros(n, dtype=int)
    path[-1] = np.argmax(delta[-1])
    for i in range(n-2, -1, -1):
        path[i] = psi[i+1][path[i+1]]
    return [idx2tag[i] for i in path]

sentence = "Bill visited California"
print("Sentence:", sentence.split())
print("Predicted NER Tags:", viterbi_ner(sentence))
```

OUTPUT:

Sentence: ['Bill', 'visited', 'California'] Predicted NER Tags: ['PER', 'O', 'LOC']
--

RESULT:

The NER system using HMM was successfully implemented and trained on a small annotated dataset. The Viterbi algorithm correctly predicted 'Bill' as PER (Person), 'visited' as O (Other), and 'California' as LOC (Location), demonstrating that the HMM learned meaningful transition and emission patterns from the training data.